

Build receipts for desktop apps

Before an app is signed, check how it was built. Every build step writes a signed receipt. The release team keeps a short signed rulebook. One command compares the two and answers PASS or BLOCKED, with the reason.

1 · BUILD TEAMS

Every step writes a receipt

Put `cilock run` in front of steps you already run. The receipt records what ran, what went in, what came out, and the result. The build machine signs it.

2 · THE RELEASE TEAM

A short signed rulebook

1. The build finished without errors.
2. The tests ran on that build and passed.
3. The package holds the exact file the build produced.

3 · BEFORE SIGNING

One check

`cilock verify` checks the receipts against the rulebook.
PASS: signing continues as normal.
BLOCKED: do not sign, and here is why.

What changes for your team

```
$ cilock run --step build -k ../keys/build-machine.key -o ../evidence/build.json --material-manifest -a git -- cc app/photo_lite.c -o photo-lite
```

Real command from the demo run. Your build command is unchanged; it goes after the `--`.

What it catches

WHAT HAPPENED	RESULT	CILOCK'S REASON (REAL OUTPUT)
The app file was replaced after the tests	BLOCKED	mismatched digests for photo-lite
The tests were skipped	BLOCKED	no collection passed verification for step test
The tests failed, and the release went ahead anyway	BLOCKED	policy was denied due to: the tests failed (exit code 3)

Built for AI agents

ROUND	CHANGE	FINDINGS	RIGHT	TIME
1	first draft	6	3/4	0.3s
2	link test + package to build	6	4/4	0.2s
3	apply cilock's suggested fix	0	4/4	0.4s

A policy is tested against known good and known bad builds. The agent stops at zero validator findings and every verdict right. `cilock`'s messages say what to fix.

What to expect

1. A 30 to 60 minute session to walk through your build steps.
2. We start with one of your pipelines.
3. Early failures are usually rule details. We adjust the rules with you.

`cilock` is a pipeline observer: GitHub Actions (`cilock-action`), GitLab CI (template), Jenkins (a normal `sh` step). The lesson passes on Linux, macOS and Windows build machines. Your own compilers and CI are confirmed during the pilot.